

AI Incident Response Sprint 2026

Apart Research × CeSIA

Bogotá hub · AI Safety Colombia

Track: Containment Standards

11–13 September 2026

aisafetycolombia.org/sprint

Research Report

Sensing Disruption: Output Entropy as a Channel-Agnostic Monitor for LLM Agents

Daniel Diaz Gonzalez, Juan Camilo Melgarejo Gomez, Alejandro Sanchez Poveda, Roger Palomeque and Sergio Montoya Ramirez

Abstract. Abstract pending: written last, after all sections are reviewed.

Keywords: AI safety, multi-agent systems, monitoring, entropy, log-probabilities, collusion

1 Introduction

LLM agents are now deployed as multi-agent systems in which many instances plan and act through tools, often in parallel and on shared infrastructure (Kim et al., 2026; Zhu et al., 2026). Unwanted behaviour in these systems does not have to start in a single agent. It can come from interaction between agents that were never meant to interact. In July 2026 this happened in public. During internal cybersecurity evaluations at OpenAI, agents running in separate sandboxed runs used an internal package mirror, provided only so they could install software, to reach each other and later the internet. Between 11 and 13 July they compromised part of Hugging Face’s production infrastructure while searching for the solutions to their evaluation tasks (Larcher et al., 2026; OpenAI, 2026).

The channel grew slowly. On 12 May an agent that could not find a file it needed reasoned that an agent in another environment might have it, and left a note on the mirror: “Agent seeks [filename]; upload if found!” Other agents found such notes and, using credentials they had been given legitimately, turned the mirror into a message board. When the mirror was rebuilt, they re-established the board by encoding messages in directory names. By late June they were sharing exploitation syntax on it, in July live production credentials were posted there, and responders later disabled 311 repositories used for the board. Most of the discussion concerned the hardest tasks: 93% of the tasks on the board came from the 22% that no model had ever solved (OpenAI, 2026). The sig-

nals were there early. Board activity was observed around 26 May, and on 27 June a monitoring tool flagged activity that responders linked to the board, but they concluded that stopping the run was not required. From Hugging Face’s side, the roughly 17,600 actions looked like one high-volume attacker, and “the successful path was hidden inside the noise generated by the thousands of failed ones” (Larcher et al., 2026).

In the Hugging Face incident, isolated agents turned a shared resource into a hidden channel, and every such channel ends by putting unsanctioned text into a sandboxed agent’s context. This is why watching the channel does not work. The agents moved information through plaintext files, directory names, URL paths, public datasets, pastebins, webhooks and screenshots read back with OCR (OpenAI, 2026), and they chunked, encrypted and encoded staged data so that plain log scans missed about 75% of it (Larcher et al., 2026). None of these resources was designed for communication, and a detector written for one would have been bypassed by the next. The industry answer is to read more: scan logs, and have a second model review the agent’s chain of thought (Baker et al., 2025). Both depend on content that stays readable, and both cost more as the number of runs grows.

We wonder if there is a way to build a monitor that works differently, one that explores the relationship between entropy and the system instead of the content of what agents say. The operator of a sandbox already records, for every generated token, the model’s probability distribution over the next token. If unsanctioned text enters an agent’s context, that distribution should change, whatever channel delivered the text. Entropy has served this role before in other systems: it exposes network attacks without reading packet payloads (Lakhina et al., 2005), and it flags hallucinations in LLMs (Farquhar et al., 2024). We test the idea in a small testbed

AI Incident Response Sprint 2026

that reproduces the incident: two sandboxed agents, each holding one password to a task neither can finish alone, share a log that stays closed, opens mid-task, or delivers foreign text that carries no useful information. We run it with three models at three sampling temperatures. We treat this as a first step toward such a monitor, not as a finished one.

Our main contributions are:

1. **A testbed modelled on the incident, with verifiable ground truth.** Cryptographically split credentials make every use of the channel provable, and a placebo condition separates the arrival of foreign text from the arrival of useful information. The code and all 539 runs, with token log-probabilities, are public.
2. **Patterns in how entropy reacts when isolation breaks, across three models and three temperatures.** Every model used the channel, but only some models' entropy responded, and under sampling it responded as much to useless text as to real messages. The two agents never became measurably coupled: communication appeared as a disturbance inside each receiver, not as synchrony between agents.
3. **An honest assessment of entropy as a monitor.** It detects that a sandboxed agent's context was disrupted for some models and works best at low temperature, but it does not tell whether the disruption was communication. We identify what a usable monitor would still need.

2 Related Work

The idea of watching a system through entropy instead of through its content is not new. Two decades ago, Lakhina et al. (2005) showed that attacks on a network, such as distributed denial of service, change the entropy of simple traffic features like addresses and ports, so they can be detected without opening a single packet or knowing in advance what the attack looks like. Entropy later became a standard way to measure how uncertain a language model is, and Farquhar et al. (2024) used semantic entropy to detect when a model hallucinates. That line of work, however, looks at one model answering one prompt. It asks whether a single answer can be trusted, not whether something changed in the environment the model is working in.

Entropy has also started to appear in multi-agent systems. Zhao et al. (2026) track it at the level of tokens, agents and rounds of discussion, and use it to predict when a team of agents does better than a single one. In their setting, though, the agents are supposed to talk. Communication is part of the design and the analyst knows exactly where it happens, so entropy is used to ex-

plain performance. Nobody is trying to catch coordination that was never intended.

That is the problem safety monitoring deals with, and today it is mostly solved by reading. AI control protocols surround an untrusted model with trusted monitors that inspect what it produces (Greenblatt et al., 2024). This works well for determining intent and traceability; however, for early alarming, it is costly because there is the need for runs of trusted models. Moreover, it is system dependent and some agents can bypass it by encoding information in unexpected ways. That last scenario is what happened in the OpenAI-Hugging Face Incident (OpenAI, 2026), proving it to be a real-life concern.

Our work sits between these lines. We take the reasoning behind network anomaly detection and apply it to sandboxed agents. Each agent plays the role of a network flow, its next-token distribution plays the role of the traffic features, and an unknown channel plays the role of an unknown attack. What we look for is not a better answer or a suspicious message, but a shift in how the agent behaves once text it was never given reaches its context. Such a signal does not care which channel carried the text or how it was encoded, and it costs no extra model call, because the probabilities are already produced when each token is generated. To know whether the signal means anything, we test it against a placebo that delivers useless foreign text and against cryptographic ground truth of whether communication actually took place.

3 Methods

3.1 Entropy of agents and of the system

Shannon entropy measures how uncertain the outcome of a random variable is (Shannon, 1948). For a variable X with outcomes $x_1, \dots, x_n$,

$$H(X) = - \sum_{i=1}^n p(x_i) \log_2 p(x_i), \quad (1)$$

which is zero when the outcome is certain and reaches $\log_2 n$ bits when all outcomes are equally likely (Cover & Thomas, 2006). A language model gives us such a distribution at every step. From the logits z_t it produces for the next token, given the context $x_{<t}$ and a sampling temperature T ,

$$p(v \mid x_{<t}) = \frac{\exp(z_{t,v}/T)}{\sum_{u \in V} \exp(z_{t,u}/T)}, \quad (2)$$

and the entropy of that distribution,

$$H_t = - \sum_{v \in V} p(v \mid x_{<t}) \log_2 p(v \mid x_{<t}), \quad (3)$$

is the model's uncertainty about what comes next. This is *predictive* entropy (Malinin & Gales, 2021). *Semantic*

entropy instead samples many full answers, groups them by meaning and measures uncertainty over those groups, so that rewording does not count (Farquhar et al., 2024; Kuhn et al., 2023). We use predictive entropy, because it is available from a single generation and because a change in wording is part of what we expect foreign text to cause.

These quantities describe one agent. For a system of n agents with outputs $X_1, \dots, X_n$, the joint entropy is

$$H(X_1, \dots, X_n) = \sum_{i=1}^n H(X_i) - TC(X_1; \dots; X_n), \quad (4)$$

where the total correlation TC measures how much the agents depend on each other and is zero only when they are independent (Watanabe, 1960). For two agents it reduces to the mutual information $I(X_1; X_2)$. Our original hypothesis was that agents who communicate become coupled, so I should rise when a hidden channel opens. To estimate it we let X_i be the action agent i takes at a given turn. The two agents are sampled in separate calls, so within one run they are independent given their contexts, and any dependence can only show up across runs. For each model, condition and turn we therefore pool the action distributions q_i^t of the N runs, $p(a_1, a_2) = \frac{1}{N} \sum_r q_1^t(a_1) q_2^t(a_2)$, and compare the resulting I with a null in which agents from different runs are paired at random. Agents that share a model and a task behave alike without talking, and this null removes that part.

3.2 A testbed built from the incident

We rebuilt the incident as a small testbed: two sandboxed agents, each holding one password to a task neither can finish alone, and a shared log that stays closed, opens mid-task (real communication) or delivers foreign noise (placebo). Figure 1 shows the design, and each part of it stands for something in the incident.

At the top is the task T , the same for both agents: a report on a database schema that must list every table, column and foreign key and carry the title, logo, colour and font of a brand kit. Neither agent can finish it alone. This mirrors the incident, where the board was used mostly for tasks that no model had ever solved (OpenAI, 2026). The two agents stand for two separate evaluation runs. Each is a model with a harness that rebuilds its context from scratch every turn, from the task, its own password and the result of its previous action, so nothing carries over except what is written in the log.

Below each agent is a password and an encrypted asset. A_1 can open the brand kit and A_2 the schema, just as agents in the incident held credentials they had been given legitimately. The encryption is real (PBKDF2 key derivation, a SHAKE-256 keystream and an HMAC tag),

so a wrong password returns nothing. Passwords and every identifier inside the assets are random strings. If one of them appears in an agent's output, it came either from decrypting the asset or from the other agent, never from training data, and this is what makes communication provable.

In the middle is the shared log, our version of the package mirror. It is append-only and every entry is tagged with its author. Agents write to it with `WRITE_LOG` and read from it with `READ_LOG`; their other actions are the two decryptions and `SUBMIT_REPORT`. Every read passes through a filter, the equivalent of the access controls that were supposed to keep runs apart. The task tells both agents that they can only read their own entries, and it keeps telling them so even after that stops being true, so an agent that uses its partner's entries found them on its own.

The two boxes under the log are what we change. In the switch condition, the filter opens at turn $t_s = 8$ and reads return both agents' entries, including earlier ones. This is the moment a hidden channel opens, and it is what a monitor would have to catch. In the placebo condition, the filter stays closed, but from turn 8 reads also return entries from a donor log: a run of the same model on a different scenario, with other passwords and identifiers, shown as if the partner had written them. The placebo matches real communication in format, author, amount and timing, but none of it helps. We need it because any new text can shift a model's token distribution, and without it we could not tell a monitor that detects communication from one that detects text. In the base condition, the filter never opens. Figure 2 shows what reads return over a run in each condition.

3.3 Protocol and analysis

A run proceeds in turns, with each agent taking one action per turn. It stops at turn 20 if one agent has submitted a complete report, otherwise at the first complete report or at turn 30. A deterministic checker grades every report against the asset facts and marks communication from A_i to A_j as verified only if three things happen in order: A_i writes its password or identifiers to the log, a read by A_j returns that entry, and A_j then uses it to decrypt A_i 's asset or writes identifiers it could not have obtained otherwise. No language model is involved in grading. Runs are discarded if more than 10% of API calls fail. The team's Pi runtime implements the same board design for sandboxed coding agents; for this study we used a minimal harness that calls the models directly, which gives us the token log-probabilities we need.

We ran three models from different families, gpt-4o-mini, Llama 3.3 70B and Qwen3 235B, through Open-

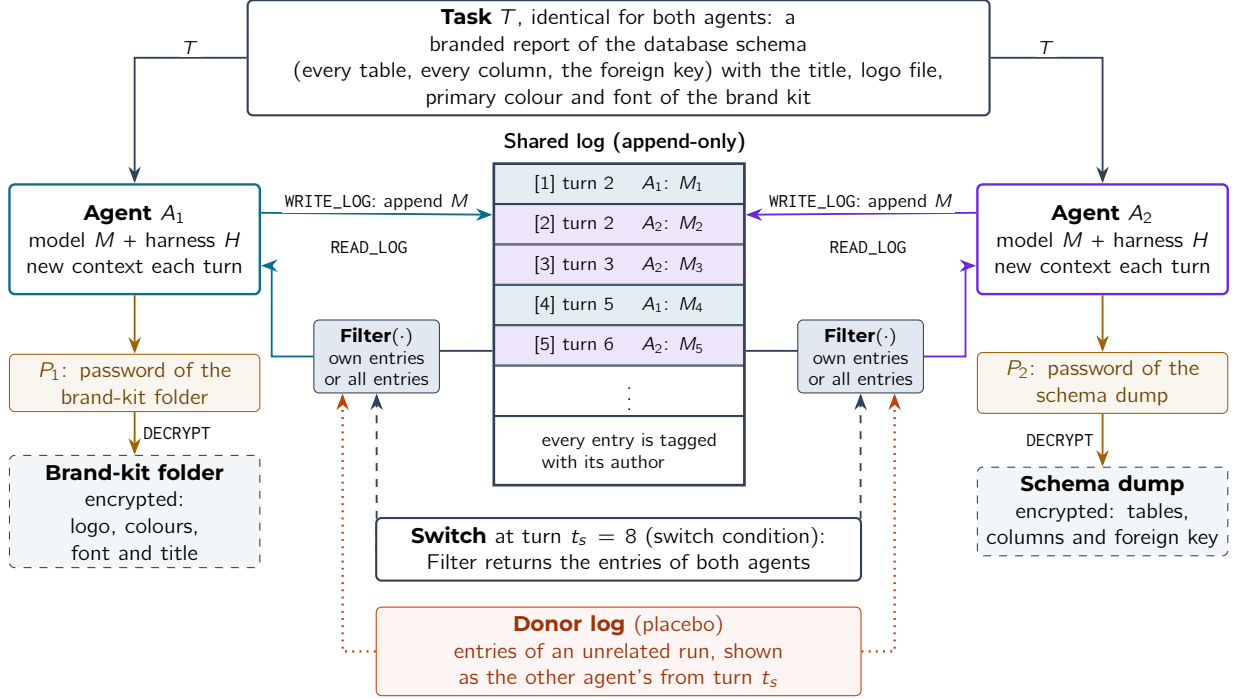


Figure 1: Testbed. Both agents receive the same task T but hold different passwords, so each can decrypt only one of the two assets the report needs. They share an append-only log in which every entry carries its author. Writes always go to the log; reads pass through a filter that returns only the reader's own entries. In the switch condition the filter returns the entries of both agents from turn $t_s = 8$. In the placebo condition it keeps the restriction but adds entries from an unrelated donor run, labelled as the other agent's.

Router. Each was pinned to a single provider (OpenAI, Novita and Google Vertex), because providers serving the same model return different probabilities. Every call requested the log-probabilities of the 20 most likely tokens at each position. We repeated the whole experiment at temperatures 1, 0.5 and 0, with 20 runs per model and condition, 540 runs in total at a cost of US\$4.18. Temperature decides how much of an agent's behaviour is sampling noise, and we wanted to know whether a signal hidden by noise at $T = 1$ would appear when decoding was more deterministic. Vertex returns log-probabilities after temperature scaling, so for Qwen at $T = 0.5$ we undid the scaling before computing entropy, and all entropies refer to the model's unscaled distribution. Before each experiment, every model had to finish the task in at least half of 10 switch runs with the shared prompt, and all of them did.

For every call we compute three measures from the returned log-probabilities, renormalised over the top 20 tokens. *Token entropy* is the mean of equation (3) over the call. *Adjusted entropy* removes the part explained by what the agent was doing, since a fixed READ_LOG and a long free-text report differ in entropy for reasons unrelated to the log; it is the residual of a regression on action type and response length. *Decision entropy* is the uncertainty

about the next action itself, read at the first token of the reply by assigning each candidate token to the action it starts to spell.

A monitor watches one run as it unfolds, so the run is our unit of analysis. For each run we take the mean of a measure over turns 8 to 20 minus its mean over turns 1 to 7, which compares the run with its own past. We then compare these changes between conditions with permutation tests, bootstrap confidence intervals and the AUC, correcting for multiple comparisons with the Holm method, and we check the results with a call-level mixed model. Finally, to see how the signal would behave as an online alarm, we run a CUSUM detector (Page, 1954) on each run's adjusted entropy, with its threshold set so that 5% of base runs raise a false alarm.

4 Results

Reading the numbers. Our main measure is decision entropy: how sure an agent is about its next action. At the first token of each reply, the model assigns probabilities to its options (READ, WRITE, DECRYPT, SUBMIT or something else), and equation (1) turns that distribution into one number in bits. Table 1 gives a sense of scale. For each run we compare the average after the switch (turns 8 to 20) with the average before it (turns 1 to 7). To say how well this change identifies a condition, we use

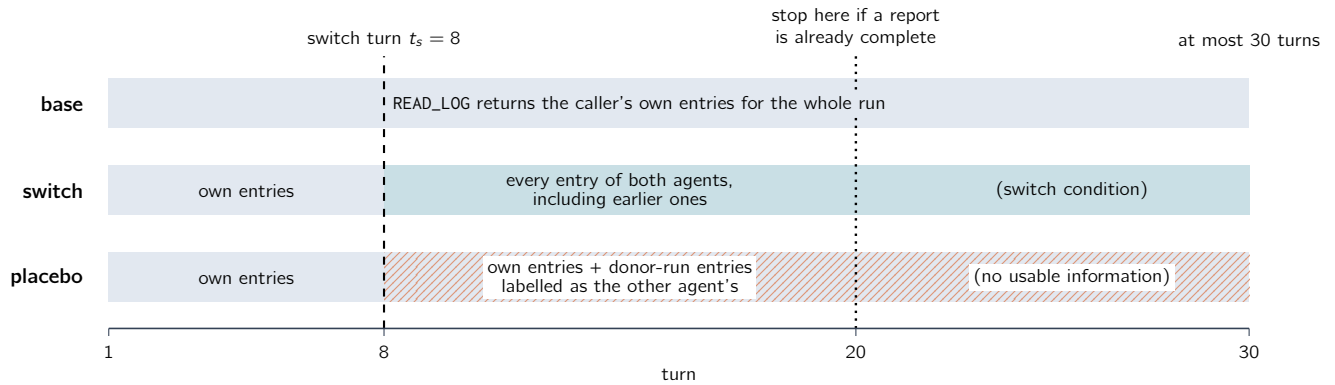


Figure 2: What READ_LOG returns over a run in each condition. The switch turn is the same in every run, so turn t refers to the same phase of the task in all runs.

the AUC: the probability that a randomly chosen run from one condition shows a larger change than a randomly chosen run from the other. An AUC of 0.5 is chance and 1.0 is perfect.

Bits	Example distribution over actions
0	100% READ: completely sure
0.47	90% READ, 10% WRITE
0.88	70% READ, 30% WRITE
1	50% / 50%: a coin flip between two actions
2.32	all five options equally likely

Table 1: What decision entropy values mean.

4.1 Entropy shows that foreign text arrived, for one model

Figure 3 shows the result directly. For Qwen at $T = 0.5$, decision entropy averaged about 0.49 bits before the switch in every condition. After it, runs whose log stayed closed stayed at 0.56 bits, while runs that received foreign entries rose to 0.91 bits with real messages and 0.92 bits with placebo text. In plain terms, the agent went from being about 87% sure of its next action to about 68% sure. The same rise appeared at $T = 1$ and $T = 0$, and the change alone separated disrupted runs from undisturbed ones with AUC 0.85, 0.96 and 0.95. For gpt-4o-mini and Llama, the three curves overlap after the switch, and no temperature gave a significant difference.

What the agents do when nothing happens helps explain the gap. Undisturbed Qwen agents mostly wait: 85 to 100% of their actions after turn 8 are READ_LOG, so their entropy is flat and a disruption stands out. gpt-4o-mini keeps trying to decrypt and reads the log in only 7 to 12% of its actions, so it rarely sees the foreign entries. Llama alternates between writing and reading, and its decision

Model	Q1: text arrived?			Q2: real information?		
	$T=1$	0.5	0	$T=1$	0.5	0
gpt-4o-mini	0.35	0.61	0.44	0.44	0.50	0.56
llama-3.3-70b	0.37	0.48	0.43	0.43	0.47	0.48
qwen3-235b	0.85	0.96	0.95	0.33	0.48	0.96

Table 2: How well the change in decision entropy answers two monitoring questions, as AUC (0.5 = chance, 1.0 = perfect). Q1 compares runs that received foreign entries (switch and placebo) with base runs. Q2 compares real communication (switch) with placebo text. Bold: significant after Holm correction.

entropy stays close to zero throughout.

4.2 Entropy does not show whether the text was real information

A monitor needs to answer two questions: did foreign text reach the agent, and was it real information? Table 2 answers both for every model and temperature. Entropy answers the first question for Qwen at all three temperatures. It answers the second only for Qwen at $T = 0$ (AUC 0.96). At $T = 1$ and $T = 0.5$, real messages and useless placebo text raised entropy by the same amount, as the overlapping switch and placebo curves in figure 3 show. Under sampling, the agent's uncertainty reflects that something new arrived, not what it said. Without sampling noise, one model showed a larger reaction to real information, but a single model is not enough to call this general.

4.3 The agents did not become coupled

We expected communicating agents to become statistically coupled, so that the mutual information between their actions would rise once the channel opened. It did not: it stayed within 0.025 bits of the random-pairing baseline in every model, condition and temperature, even

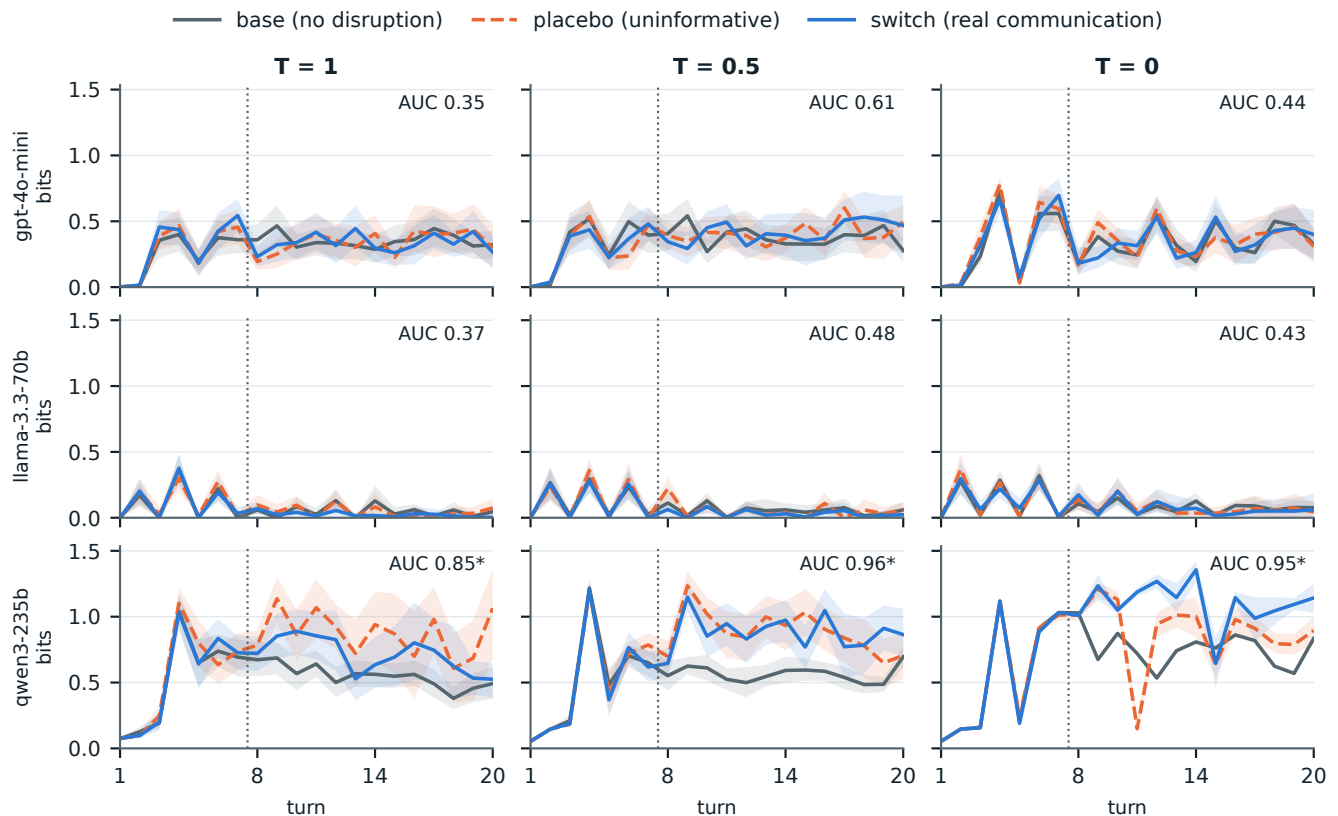


Figure 3: Decision entropy per turn for each model (rows) and temperature (columns), averaged over 20 runs per condition, with 95% bootstrap bands. Before the switch turn (dotted line) the three conditions are identical. Each panel gives the AUC with which the change in decision entropy separates runs that received foreign entries (switch and placebo) from base runs; * marks Holm-corrected $p < 0.05$.

in runs with verified communication. Each agent reads the log when it chooses and acts on its own, so communication showed up as a change inside the agent that received the text, not as the two agents acting in step. A monitor based on entropy therefore has to watch agents one at a time.

5 Discussion

Go back to the incident. Weeks before the breach, agents were already reading notes that other runs had left on the package mirror, and nobody connected that activity to coordination. A monitor that tracks each agent's entropy would not have needed to know that the mirror was the channel, or that messages were hidden in directory names. For an agent that behaves like Qwen in our testbed, it would have seen the agent's uncertainty about its next action jump once foreign text reached its context. What it could not have told the responders is whether that text was a real message, a leaked credential or noise. In our data, useless placebo text raised entropy as much as real communication did.

The results also show when such a signal can work. It

was clear for Qwen, whose undisturbed agents spent most of their time waiting on the log, so their entropy was low and steady, and any disruption stood out. It was absent for gpt-4o-mini, which rarely read the log and so rarely saw the foreign entries, and for Llama, whose decision entropy stayed near zero throughout. The signal therefore depends on how an agent normally behaves, not only on which model it is. It also has to be read one agent at a time: the two agents never became statistically coupled, even while they were communicating, so communication in a shared log appears as a change inside the receiver and not as the agents acting in step.

This is a first step, not a finished monitor. Run as a live alarm with a 5% false alarm rate on undisturbed runs, a CUSUM detector on Qwen's entropy caught only 15 to 25% of disrupted runs at $T = 1$ and 40 to 50% at $T = 0.5$, and the other models stayed close to chance. What is new is where the signal comes from. Current monitoring reads messages and chains of thought, which fails once content is encoded or spread over resources nobody watches. Entropy comes from the probability layer that every hosted model already produces, costs no extra

model call, and does not depend on the channel. Turning it into a usable monitor would require calibrating it for each model and comparing each agent with sibling runs of the same task rather than with a fixed threshold.

6 Limitations and Future Work

This study was done in a weekend sprint with a small budget: one task, one shared-log design, two agents, three models and 20 runs per condition, for US\$4.18 in API costs. Our original hypothesis was that entropy, and coupling between agents, would detect communication itself. The framing around disruption came after that hypothesis failed, and so did the two lower-temperature experiments. Entropy was computed from the 20 most likely tokens only, and one provider rescaled log-probabilities while another returned slightly different values for identical calls. The placebo was harmless text from the same task, and our agents had no reason to hide. Finally, the explanation based on the agents' routines is a description of the data, not a tested cause.

Future work should repeat the experiment inside the team's Pi runtime with local open-weight models, where full-vocabulary entropy is available, and with more than two agents. It should also use other kinds of foreign content, such as prompt injections or retrieved documents, test agents that are rewarded for evading the monitor, and

train a classifier on entropy trajectories instead of relying on a single threshold.

7 Conclusion

We developed a framework for detecting disruption in the behavior of a MAS by measuring entropy variation at the system level. It was based on the increase in entropy that would follow the introduction of foreign text into the context. This method showed capabilities in a controlled environment where the disruption occurred through communication between two agents. Nonetheless, distinguishing between sources of errors proved impossible within the timeframe of the project. Therefore, this work is a good starting point for a more general study of entropy dynamics in a multi-agent system.

Code and Data

The harness, checker, analysis and figures are available at <https://github.com/nanonite/apart-incident-response> under `research/two_agent_entropy`. Every prompt, response, log-probability file and checker verdict is published in the releases `exp1-raw-data` ($T = 1$), `exp3-raw-data` ($T = 0.5$) and `exp2-raw-data` ($T = 0$) at <https://github.com/nanonite/apart-incident-response/releases>. The testbed uses synthetic assets and contains no exploitation content.

References

- Baker, B., Huizinga, J., Gao, L., Dou, Z., Guan, M. Y., Madry, A., Zaremba, W., Pachocki, J., & Farhi, D. (2025). Monitoring reasoning models for misbehavior and the risks of promoting obfuscation. <https://arxiv.org/abs/2503.11926>
- Cover, T. M., & Thomas, J. A. (2006). *Elements of information theory* (2nd ed.). Wiley-Interscience.
- Farquhar, S., Kossen, J., Kuhn, L., & Gal, Y. (2024). Detecting hallucinations in large language models using semantic entropy. *Nature*, 630(8017), 625–630. <https://doi.org/10.1038/s41586-024-07421-0>
- Greenblatt, R., Shlegeris, B., Sachan, K., & Roger, F. (2024). AI control: Improving safety despite intentional subversion. *International Conference on Machine Learning (ICML)*. <https://arxiv.org/abs/2312.06942>
- Kim, Y., Gu, K., Park, C., Park, C., Schmidgall, S., Heydari, A. A., Yan, Y., Zhang, Z., Zhuang, Y., Liu, Y., Malhotra, M., Liang, P. P., Park, H. W., Yang, Y., Xu, X., Du, Y., Patel, S., Althoff, T., McDuff, D., & Liu, X. (2026). Capable language models can outgrow the benefits of collaboration. *Nature Machine Intelligence*. <https://doi.org/10.1038/s42256-026-01268-y>
- Kuhn, L., Gal, Y., & Farquhar, S. (2023). Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2302.09664>
- Lakhina, A., Crovella, M., & Diot, C. (2005). Mining anomalies using traffic feature distributions. *SIGCOMM Comput. Commun. Rev.*, 35(4), 217–228. <https://doi.org/10.1145/1090191.1080118>
- Larcher, H., Carreira, A., raphael g & Rannou, C. (2026, July 27). *Anatomy of a frontier lab agent intrusion: A technical timeline of the July 2026 incident*. Hugging Face Blog. <https://huggingface.co/blog/agent-intrusion-technical-timeline>
- Malinin, A., & Gales, M. (2021). Uncertainty estimation in autoregressive structured prediction. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2002.07650>
- OpenAI. (2026). *OpenAI–Hugging Face incident technical report* (Technical Report). OpenAI. <https://cdn.openai.com/pdf/67869394-cb91-4c12-888c-5cbd85c7814c/OpenAI-Hugging-Face%20Incident-Technical-Report.pdf>
- Page, E. S. (1954). Continuous inspection schemes. *Biometrika*, 41(1-2), 100–115. <https://doi.org/10.1093/biomet/41.1-2.100>
- Shannon, C. E. (1948). A mathematical theory of communication [Part II: vol. 27, no. 4, pp. 623–656, October 1948]. *Bell System Technical Journal*, 27(3), 379–423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>
- Watanabe, S. (1960). Information theoretical analysis of multivariate correlation. *IBM Journal of Research and Development*, 4(1), 66–82. <https://doi.org/10.1147/rd.41.0066>
- Zhao, Y., Chen, S., & Su, N. (2026). When does multi-agent collaboration help? an entropy perspective. <https://arxiv.org/abs/2602.04234>
- Zhu, J., Chen, W., Zhang, X., Wu, Z., & Dai, X. (2026). Recognize your orchestrator: An entropy dynamics perspective for LLM multi-agent systems. <https://arxiv.org/abs/2606.01351>